

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу
«Операционные системы»

Группа: М8О-216БВ-24

Студент: Ермакова М. П.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.11.25

Москва, 2025

Постановка задачи

Вариант 17.

Найти в большом целочисленном массиве минимальный и максимальный элементы

Общий метод и алгоритм решения

Использованные системные вызовы:

- `void *sbrk(intptr_t increment);` – выделяет память под массив.
- `int gettimeofday(struct timeval *tv, struct timezone *tz);` – получает текущее время с точностью до микросекунд.
- `ssize_t write(int fd, const void *buf, size_t count);` – вывод данных.
- `pthread_t pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);` – создает новый поток выполнения.
- `int pthread_join(pthread_t thread, void **retval);` – ожидает завершения указанного потока и освобождает его ресурсы.
- `int pthread_mutex_lock(pthread_mutex_t *mutex);` – блокирует мьютекс, обеспечивая эксклюзивный доступ к критической секции кода.
- `int pthread_mutex_unlock(pthread_mutex_t *mutex);` – освобождает ранее заблокированный мьютекс, позволяя другим потокам получить к нему доступ.

В рамках лабораторной работы была разработана программа, способная обрабатывать данные в многопоточном режиме. Программа создает массив заданного размера, заполняет его псевдослучайными числами и выполняет поиск минимума и максимума двумя способами, для сравнения последовательного и параллельного алгоритмов поиска необходимых значений в большом массиве целых чисел.

Сначала работает последовательная версия, которая в одном потоке проходит по всему массиву и находит искомые элементы. Затем запускается параллельная версия, где массив разделяется между несколькими потоками - каждый поток обрабатывает свой участок массива и находит локальные минимум и максимум, после чего главный поток собирает все результаты и определяет глобальные значения.

Программа выводит время работы каждого способа обработки, что демонстрирует ускорение работы при использовании многопоточности и позволяет исследовать зависимость производительности от количества потоков.

Код программы

parallel.c

```
#include <pthread.h>

#include <time.h>

#include <limits.h>

#include <unistd.h>

#include <sys/time.h>

#include <stdlib.h>
```

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int global_min = INT_MAX;

int global_max = INT_MIN;
```

```
typedef struct {

    int *array;

    int start;

    int end;

    int min;

    int max;

    int id;

} thread_data_t;
```

```
void write_number(int num) {

    char buffer[100];
```

```
    int i = 0;

    int sign = 1;

    if (num < 0) {

        sign = -1;

        num = -num;

    }

    do {

        buffer[i++] = '0' + (num % 10);

        num /= 10;

    } while (num > 0);

    if (sign == -1) {

        buffer[i++] = '-';

    }

    for (int j = 0; j < i/2; j++) {

        char temp = buffer[j];

        buffer[j] = buffer[i - j - 1];

        buffer[i - j - 1] = temp;

    }
```

```
buffer[i] = '\n';

write(STDOUT_FILENO, buffer, i + 1);
```

```
}
```

```
int my_strlen(const char* str) {
```

```
    const char* str_copy = str;

    int len = 0;
```

```
    while(*str_copy != '\0') {

        len++;

        str_copy++;

    }

    return len;

}
```

```
unsigned long my_atoi(const char* number) {
```

```
    const char* str = number;

    unsigned long res = 0;
```

```
    while (*str != '\0') {

        char digit = *str;

        res = res * 10 + (digit - '0');

        str++;

    }
```

```
    return res;

}
```

```
int* my_random(int* array, unsigned long size) {
```

```
    if (!array || size == 0) return array;
```

```
    struct timeval tv;

    gettimeofday(&tv, NULL);

    int next = tv.tv_sec % 1000000;
```

```
    for (unsigned long i = 0; i < size; i++) {

        next = ((next * 214013 + 2531011) % 2147483648) % 1000000;

        array[i] = next;
```

```
    }
```

```
    return array;
}
```

```
void* find_min_max(void* arg) {
```

```
    thread_data_t* data = (thread_data_t*)arg;

    int local_min = INT_MAX;

    int local_max = INT_MIN;
```

```
    for (int i = data->start; i < data->end; i++) {

        if (data->array[i] > local_max) local_max = data->array[i];

        if (data->array[i] < local_min) local_min = data->array[i];

    }
```

```
    pthread_mutex_lock(&mutex);

    if (local_min < global_min) global_min = local_min;

    if (local_max > global_max) global_max = local_max;

    pthread_mutex_unlock(&mutex);
```

```
    return NULL;
}
```

```
long long time_now() {

    struct timeval tv;
```

```
    gettimeofday(&tv, NULL);

    return (long long)tv.tv_sec * 1000000LL + tv.tv_usec;
}
```

```
int main(int argc, char* argv[]) {
```

```
    size_t size = my_atoi(argv[1]);
```

```
    int* array = (int*)sbrk(size * sizeof(int));

    if (array == (void *)-1) {

        char* str = "Memory allocation error!\n";

        write(STDERR_FILENO, str, my_strlen(str));

        return 1;

    }

    my_random(array, size);
```

```
    thread_data_t one_thread;

    one_thread.array = array;

    one_thread.start = 0;

    one_thread.end = size;
```

```
    global_min = INT_MAX;

    global_max = INT_MIN;

    one_thread.id = 0;
```

```
    long long start_time = time_now();

    find_min_max(&one_thread);

    long long consistent_time = time_now() - start_time;
```

```
    write(STDOUT_FILENO, "Sequential - Min: ", 18);

    write_number(global_min);

    write(STDOUT_FILENO, "Max: ", 5);

    write_number(global_max);

    write(STDOUT_FILENO, "Time: ", 6);

    write_number(consistent_time);

    write(STDOUT_FILENO, "\n", 1);
```

```
int thread_counter = my_atoi(argv[2]);
```

```
if (thread_counter > 1) {  
    pthread_t threads[thread_counter];  
    thread_data_t thread_data[thread_counter];
```

```
global_min = INT_MAX;  
global_max = INT_MIN;
```

```
size_t size_part = size / thread_counter;  
size_t ost_size = size % thread_counter;  
int cur_start = 0;
```

```
for (int i = 0; i < thread_counter; i++) {  
    thread_data[i].array = array;  
    thread_data[i].start = cur_start;  
    thread_data[i].end = cur_start + size_part + (i < ost_size ? 1 : 0);
```

```
    cur_start = thread_data[i].end;  
}
```

```
start_time = time_now();
```

```
for (int i = 0; i < thread_counter; i++) {  
    pthread_create(&threads[i], NULL, find_min_max, &thread_data[i]);  
}
```

```
for (int i = 0; i < thread_counter; i++) {  
    pthread_join(threads[i], NULL);  
}
```

```
long long parallel_time = time_now() - start_time;
```

```
write(STDOUT_FILENO, "Parallel - Min: ", 16);

write_number(global_min);

write(STDOUT_FILENO, "Max: ", 5);

write_number(global_max);

write(STDOUT_FILENO, "Time: ", 6);

write_number(parallel_time);

write(STDOUT_FILENO, "\n", 1);
```

```
}
```

```
return 0;
```

```
}
```

Протокол работы программы

Тестирование:

Тест №1

```
Parallel - Min: -997410
Max: 999591
Time: 3437
```

Тест №2

```
many@ManyBook:/mnt/d/LAB2/OS/laba2$ ./parallel 1000000 4
Sequential - Min: -998251
Max: 999591
Time: 3418

Parallel - Min: -998251
Max: 999591
Time: 1802
```

Тест №3


```

many@ManyBook:/mnt/d/LAB2/OS/laba2$ ./parallel 1000000 6
Sequential - Min: -997410
Max: 999591
Time: 3269

Parallel - Min: -997410
Max: 999591
Time: 1590

```

Тест №5

```

many@ManyBook:/mnt/d/LAB2/OS/laba2$ ./parallel 1000 4
Sequential - Min: -998251
Max: 999591
Time: 4

Parallel - Min: -998251
Max: 999591
Time: 432

```

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	3167	1	1
2	2304	1,37	0,685
3	1873	1,69	0,563
4	1528	2,07	0,518
5	1518	2,08	0,416

Ускорение показывает во сколько раз применение параллельного алгоритма уменьшает время решения задачи по сравнению с последовательным алгоритмом. Ускорение определяется величиной $S_N = T_1/T_N$, где T_1 – время выполнения на одном потоке, T_N – время выполнения на N потоках.

Эффективность – величина $E_N = S_N/N$, где S_N – ускорение, N – количество используемых потоков.

График зависимости ускорения от количества потоков:

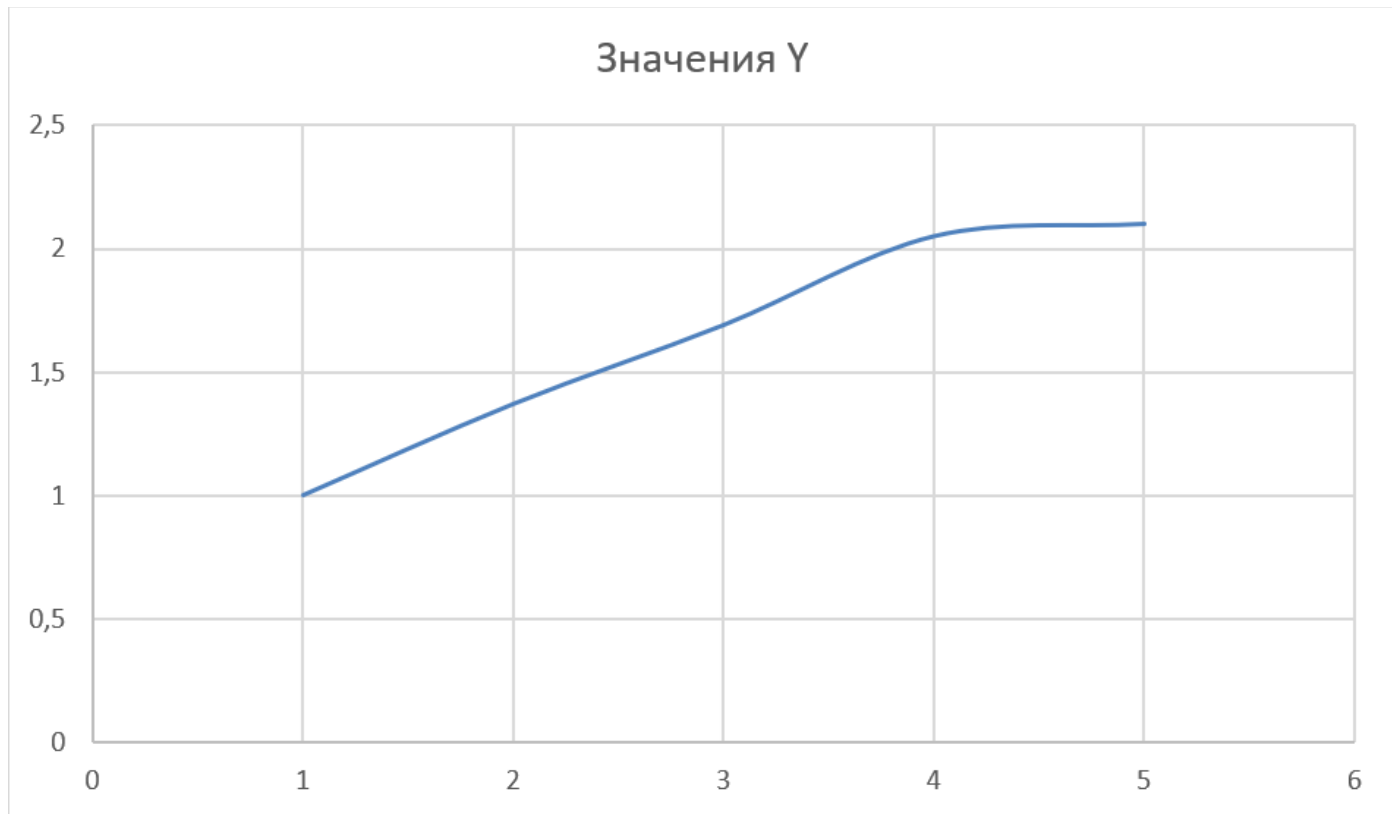
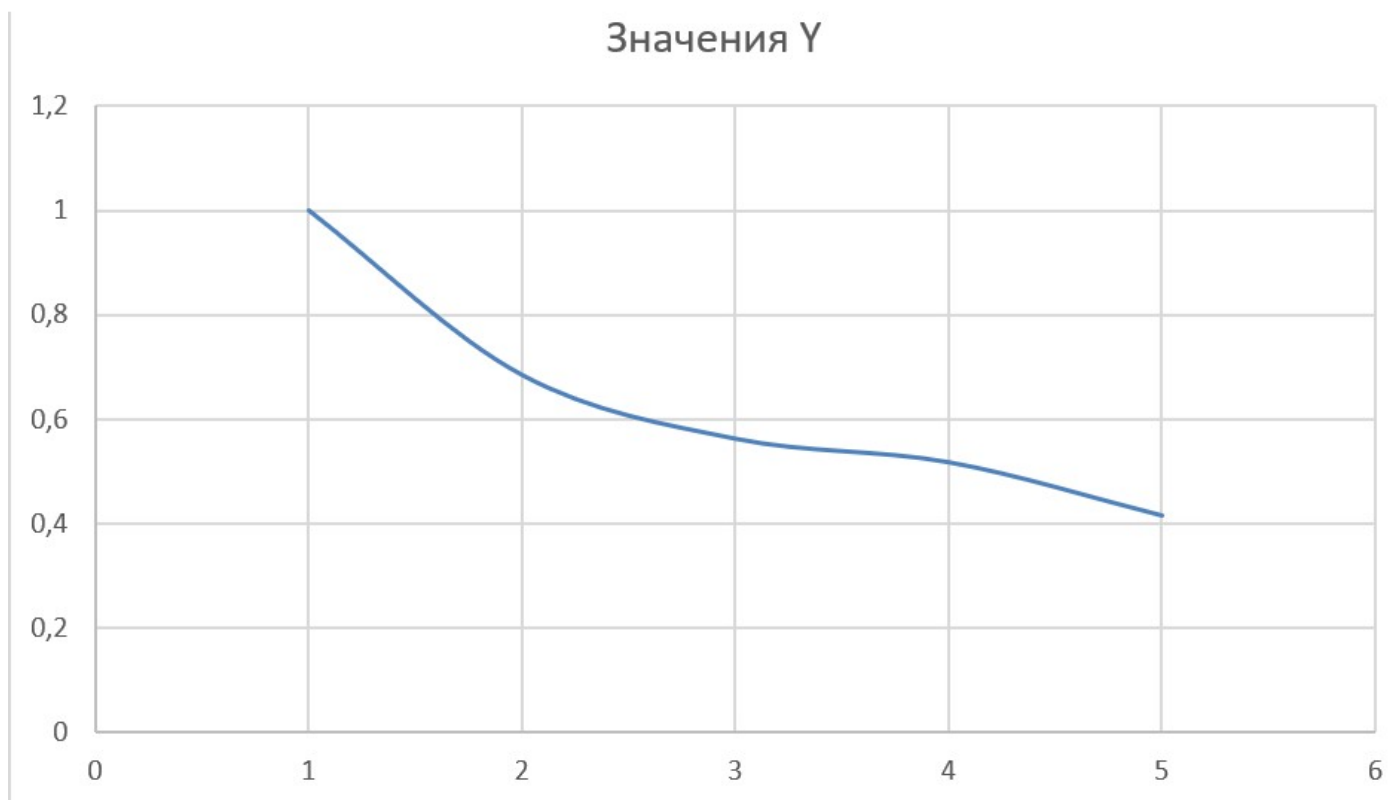


График зависимости эффективности от числа потоков:



Полученные результаты показывают умеренную эффективность распараллеливания. Такие результаты характерны для задач с частой синхронизацией между потоками, где затраты на блокировки мьютексов и обновление общих переменных значительно снижают выгоду от параллелизма. Низкая эффективность указывает на то, что большая часть процессорного времени тратится не на полезные вычисления, а на управление потоками и их взаимодействие.

Содержимое strace:

```
execve("./parallel", [".parallel", "1000000", "4"], 0x7ffc4b96e600 /* 26 vars */) = 0
brk(NULL)                               = 0x55f37398f000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f54feff8000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=19527, ...}) = 0
mmap(NULL, 19527, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f54feff3000
close(3)                                 = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f54fede1000
mmap(0x7f54fee09000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f54fee09000
mmap(0x7f54fef91000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x7f54fef91000
mmap(0x7f54fefe0000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7f54fefe0000
mmap(0x7f54fefe6000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f54fefe6000
close(3)                                 = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f54fedde000
arch_prctl(ARCH_SET_FS, 0x7f54fedde740) = 0
```

```

set_tid_address(0x7f54feddea10)      = 1061
set_robust_list(0x7f54feddea20, 24)  = 0
rseq(0x7f54feddf060, 0x20, 0, 0x53053053) = 0
mprotect(0x7f54fef0000, 16384, PROT_READ) = 0
mprotect(0x55f340f6b000, 4096, PROT_READ) = 0
mprotect(0x7f54ff030000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0

munmap(0x7f54feff3000, 19527)        = 0
brk(NULL)                            = 0x55f37398f000
brk(0x55f373d5f900)                  = 0x55f373d5f900
write(1, "Sequential - Min: ", 18)    = 18
write(1, "-997410\n", 8)              = 8
write(1, "Max: ", 5)                  = 5
write(1, "999997\n", 7)               = 7
write(1, "Time: ", 6)                 = 6
write(1, "4209\n", 5)                 = 5
write(1, "\n", 1)                     = 1

rt_sigaction(SIGRT_1, {sa_handler=0x7f54fee7a530, sa_mask=[], sa_flags=SA_RESTORER|
SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7f54fee26330}, NULL, 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0) = 0x7f54fe5dd000
mprotect(0x7f54fe5de000, 8388608, PROT_READ|PROT_WRITE) = 0
getrandom("\x05\x55\xb9\xa8\x23\x0f\x61\x7a", 8, GRND_NONBLOCK) = 8
brk(0x55f373d80900)                   = 0x55f373d80900
brk(0x55f373d81000)                   = 0x55f373d81000
rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
CLONE_CHILD_CLEARTID, child_tid=0x7f54feddd990, parent_tid=0x7f54feddd990,

```

exit_signal=0, stack=0x7f54fe5dd000, stack_size=0x7fff80, tls=0x7f54fedddd6c0} =>
{parent_tid=[1062]}, 88) = 1062

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0) = 0x7f54fdddc000

mprotect(0x7f54fdddc000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
CLONE_CHILD_CLEAR_TID, child_tid=0x7f54fe5dc990, parent_tid=0x7f54fe5dc990,
exit_signal=0, stack=0x7f54fdddc000, stack_size=0x7fff80, tls=0x7f54fe5dc6c0} =>
{parent_tid=[1063]}, 88) = 1063

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0) = 0x7f54fd5db000

mprotect(0x7f54fd5dc000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
CLONE_CHILD_CLEAR_TID, child_tid=0x7f54fdddb990, parent_tid=0x7f54fdddb990,
exit_signal=0, stack=0x7f54fd5db000, stack_size=0x7fff80, tls=0x7f54fdddb6c0} =>
{parent_tid=[1064]}, 88) = 1064

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0) = 0x7f54fcdda000

mprotect(0x7f54fcddb000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|
CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|
CLONE_CHILD_CLEAR_TID, child_tid=0x7f54fd5da990, parent_tid=0x7f54fd5da990,
exit_signal=0, stack=0x7f54fcdda000, stack_size=0x7fff80, tls=0x7f54fd5da6c0} =>
{parent_tid=[1065]}, 88) = 1065

rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

write(1, "Parallel - Min: ", 16) = 16

write(1, "-997410\n", 8) = 8

write(1, "Max: ", 5) = 5

write(1, "999997\n", 7) = 7

write(1, "Time: ", 6) = 6

write(1, "16568\n", 6) = 6

write(1, "\n", 1) = 1

exit_group(0) = ?

+++ exited with 0 +++

Вывод

В ходе лабораторной работы было установлено, что многопоточная обработка данных обеспечивает значительное ускорение вычислений при работе с большими массивами. При этом оптимальная производительность достигается при количестве потоков, соответствующем количеству физических ядер процессора. Также в ходе тестирования было доказано, что для малых объемов данных многопоточность оказывается неэффективной из-за значительных накладных расходов на создание и синхронизацию потоков.